

Mobile Application Privacy Risk Assessments from User-authored Scenarios: Replication, Refactoring, and Hardware Acceleration

Songyu Li(李松峪)

1220018776

Macau University of Science and

Technology

Macau SAR

ozakikinji55@gmail.com

Abstract—Contemporary mobile applications frequently collect extensive amounts of personal user data. However, due to a general lack of legal expertise and specific privacy resources among development teams, inadvertent privacy abuses and subsequent legal disputes routinely arise after application deployment. To address this challenge, a scenario-driven privacy risk assessment[1] method has been proposed to elicit fine-grained user privacy preferences directly from user-authored scenarios. During the subsequent baseline evaluation and model replication process, this study identified substantial bottlenecks regarding low training efficiency, disorganized software dependency chains, and chaotic file management structures within the initial implementation. To overcome these limitations, an engineering refactoring strategy is presented, migrating the environment into a standard production-ready structure and successfully introducing hardware acceleration support via CUDA and Apple Silicon's Metal Performance Shaders (MPS). The experimental results demonstrate that the proposed refactoring substantially improves training throughput and mitigates structural errors, thereby offering a more scalable implementation framework for automating user privacy risk assessments.

Keywords—computational performance, CUDA, entity extraction, hardware acceleration, privacy, requirements, risk, scenarios.

I. INTRODUCTION

With the rapid development of mobile technology, contemporary mobile applications (apps) have disrupted traditional service paradigms but simultaneously introduced profound privacy challenges. Crucially, modern mobile apps routinely collect vast amounts of granular personal information from end-users to support core functionalities. While data collection expands exponentially, many software development teams—particularly those in startups, open-source communities, or resource-constrained environments—suffer from a severe lack of legal expertise and specific privacy engineering resources. Consequently, software developers often do not know which data types or information entities the user genuinely cares about or considers sensitive within a specific operational context. This cognitive gap between developer assumptions and user privacy expectations creates significant blind spots during early system design. As a direct result, severe privacy abuses and complex legal disputes routinely occur after the application has been released to commercial app stores.

In an effort to mitigate these regulatory and legal dilemmas, traditional paradigms primarily rely on platform-level permission control architectures and pre-installation disclosures. However, empirical software engineering field studies reveal that these conventional countermeasures exhibit critical defense deficiencies. On one hand, mobile

platform permission groups (such as those in the Google Play Store) typically cover a restricted boundary of hardware-level triggers, such as location services or contact access. They consistently fail to monitor or govern fine-grained semantic entities, such as in-app search queries or behavioral records, which users frequently consider highly sensitive. On the other hand, the vast majority of consumers do not read lengthy privacy policies prior to application initialization, rendering legal documents structurally ineffective in safeguarding data subjects. Furthermore, relying on retrospective monitoring mechanisms, such as auditing user review feedback post-release, introduces severe temporal latency. Discovering systemic privacy vulnerability through user complaints is inherently a reactive approach; it is fundamentally too late to mitigate the irreparable user data leaks and corporate financial losses that have already transpired.

To mitigate these post-production legal risks, traditional platform-level solutions, such as static privacy policies and binary app store permission frameworks, have been widely deployed. However, these approaches are frequently limited by lower user engagement and a narrow coverage of informational contexts. A promising countermeasure introduced by Huang et al. is a scenario-driven privacy risk assessment methodology, which leverages named entity recognition (NER) models to systematically extract sensitive data types from user-authored text scenarios. While this automated paradigm provides a lightweight strategy for requirements engineering, evaluating its baseline implementation reveals severe systemic bottlenecks regarding model training throughput, chaotic workspace structures, and unoptimized dependency graphs.

To close this gap, this report presents a comprehensive replication and engineering optimization study. The primary contributions of this work are twofold: first, we systematically refactor the chaotic pipeline into an isolated, production-grade project hierarchy; second, we introduce dedicated hardware acceleration routines using CUDA and Apple Silicon's Metal Performance Shaders (MPS) to resolve the underlying performance inefficiencies.

II. CORE METHODOLOGY

To construct an automated, user-centric alternative to rigid permission systems, the methodology formalized by Huang et al. establishes a pseudo-automated pipeline that transforms semantic user narratives into multi-dimensional privacy risk indicators. As illustrated in the paradigm's conceptual architectural workflow, the framework is engineered around three interconnected technical vectors: scenario elicitation, advanced sequence labeling via

Transformer-based Named Entity Recognition (NER), and contextualized risk scoring loops.

A. Contextualized Scenario Elicitation

The initialization of the evaluation workflow mandates the selection of representative mobile application user interfaces. Users are presented with application screenshots and instructed to perform a localized data masking procedure, overlaying structural redaction blocks onto highly specific identifiers (e.g., profile imagery or financial account fields) to validate the authentic interaction topology. While interacting with the interface, the data subject responds to automated prompts by authoring a comprehensive descriptive scenario of a minimum of 150 words. The resulting unstructured textual corpus must explicitly delineate three relational facets: the user's localized task goals, the sequential in-app actions executed, and the precise variables required by the application backend to render the view.

B. Named Entity Recognition (NER) Approaches

Once the unstructured scenario corpus is compiled, the text streams are ingested by natural language processing components to extract privacy-related information types. To convert these free-form narratives into predictable variables, the underlying methodology relies on Named Entity Recognition (NER). Instead of relying on rigid, regular-expression-based rules that fail to generalize across diverse user vocabularies, the framework introduces a comparative evaluation between two fundamentally distinct computational paradigms: a sequence-optimized Conditional Random Field (CRF) model and a fine-tuned Bidirectional Encoder Representations from Transformers (BERT) architecture.

- **Conditional Random Fields (CRF):** As formalized by Lafferty et al. [2], a CRF is a discriminative class of statistical graphical models dedicated to sequence labeling tasks. The core attribute of a CRF is its ability to model global conditional probabilities across an entire sequence of labels, factoring in the contextual relationships of neighboring words (e.g., recognizing that "number" following "phone" forms a unique entity). However, a baseline CRF is feature-dependent; it demands extensive manual engineering to map out linguistic properties, such as lowercase variations, sub-string prefixes, and Part-of-Speech (POS) tags extracted via SpaCy.

- **Bidirectional Encoder Representations from Transformers (BERT):** Introduced by Devlin [3] et al., BERT represents the modern state-of-the-art in neural language modeling. Utilizing dense multi-head self-attention mechanisms, BERT reads an entire text span bidirectionally, automatically constructing deep contextual representations of tokens without requiring hand-crafted features. The model is first pre-trained on massive multi-billion-word corpora to inherit a generalized understanding of syntax, and is subsequently fine-tuned via a specialized sequence classification layer to predict localized entity boundaries.

The extracted tokens are categorized into standard BIO-tags (Beginning, Inside, Outside) and mapped into three distinct semantic granularities based on user expression styles.

- **Simple Entities (SIM):** Standard, independent noun phrases that capture straightforward data elements (e.g., "phone number", "age").

- **Complex Entities (COM):** Informational types integrated within extensive clausal modifications or prepositional phrases (e.g., "money in my account").

- **Question Entities (QUE):** Epistemic boundaries or user uncertainty constraints formulated via active wh-clauses (e.g., "whether there is an accident").

C. Rationale for Model Selection and Risk Framework

The structural selection of a traditional CRF alongside a deep Transformer network serves a critical requirements engineering purpose. In software engineering text mining, developers face an intrinsic trade-off between model precision and semantic generalizability.

The CRF model is chosen due to its high computational efficiency, low hardware overhead, and robust performance on highly structured, predictable noun phrases (SIM). Because it decodes sequences based on explicit POS patterns, it yields exceptional token-level precision and operates effectively as a lightweight control baseline. Conversely, BERT is selected to resolve the fluid, unstructured, and often ungrammatical nature of user-authored scenarios. Because users express software interactions using varied linguistic sub-structures (such as conditional questions or descriptive clauses), BERT's deep attention matrices are uniquely equipped to maintain a high recall profile, capturing complex (COM) and question-based (QUE) data entities that rule-bound models consistently overlook.

Once these multi-tier labels are resolved and filtered via an automated post-processing layer to eliminate floating conjunctions and false positives, the refined entities are formatted into contextual risk surveys. Perceived privacy risk is mathematically operationalized as the psychological resistance of an individual to share their extracted data assets with an unverified third party for unrestricted intentions. Users score their vulnerability across a standard six-point semantic scale, extending from "Very Willing" (quantified as 0.0) to "Very Unwilling" (quantified as 5.0). Higher cumulative rankings (3.0 and above) alert software engineering teams to systemic user anxiety, allowing developers to systematically prioritize post-production privacy-by-design mitigations.

III. REFACTORING AND HARDWARE ACCELERATION

A. Baseline Replication and Experimental Environment

To rigorously evaluate the scenario-driven privacy risk assessment methodology proposed by Huang et al., a localized software replication pipeline was constructed using the empirical 300-scenario training corpus. The local evaluation workstation architecture is powered by an Apple Silicon M5 Pro computing system configured with 48GB of unified memory. System dependencies and execution spaces were managed via a virtualized Conda environment topology running Python 3.9.25 on a Unix-based macOS architecture. Computational natural language processing elements were orchestrated using the SpaCy v3.5 linguistic parsing suite for tokenization and localized Part-of-Speech tagging. While the sequence-level boundaries were successfully evaluated using the strict CoNLL-style metrics provided by the seqeval library, the initial attempts at scalable model retraining exposed critical software engineering vulnerabilities regarding project layout, workspace maintenance, and execution throughput.

B. Identification of Production-Grade System Deficiencies

During the implementation and scaling phases of the replication lifecycle, three systemic engineering bottlenecks were identified within the baseline repository structure:

- **Messy and Uncapped Dependencies:** The initial source tree lacked a strictly defined dependency management manifest. Missing library constraints led to intense dependency graph entanglement, causing library mismatches and silent runtime errors during downstream text preprocessing.
- **Chaotic File Architecture:** The source code configuration was severely uncoupled, scattering algorithmic parameters across heavily nested subdirectories. This layout blocked standard integration testing practices and degraded long-term maintenance cycles.
- **Low Training Efficiency via Interactive Notebooks:** The core training logic for the BERT Transformer layer was natively hosted inside interactive Jupyter Notebook structures (e.g., .ipynb format). While suitable for isolated exploratory tracking, hosting large-scale language model fine-tuning inside notebook runtimes introduces significant memory usage overhead, lacks headless automation interfaces, and fundamentally restricts training throughput under large iteration spaces.

C. Production-Ready Refactoring

To mitigate the architectural vulnerabilities described above, the workspace was systematically refactored into an isolated, production-grade project hierarchy. First, a comprehensive requirements.txt file was explicitly compiled, freeze-locking compatible versions for torch==2.0.1, transformers==4.30.2, datasets==2.13.0, and accompanying NLP parsing toolkits to achieve deterministic runtime behaviors across divergent execution platforms. Second, the fragmented interactive codebase was unified and fully migrated out of Jupyter Notebook formats into decoupled, modular .py software assets (e.g., train.py, eval.py). This layout decoupling eliminates runtime serialization state overhead and exposes standard command-line flags to support seamless headless batch processing workflows.

D. Introduction of Hardware-Aware Acceleration (CUDA and MPS)

The computational core of the sequence labeling pipeline relies on fine-tuning a deep BERT layer. By default, the baseline Hugging Face TrainingArguments route sequences back to single-threaded CPU execution. This CPU fallback introduces severe compute bottlenecks during the matrix multiplications inside the multi-head self-attention backward pass. To optimize training throughput without degrading phrase-level accuracy metrics, we integrated a hardware-aware backend switch directly into the modeling runtime.

The acceleration layout is implemented via PyTorch tensor bindings. During the model initialization phase, the runtime execution context checks the host environment to detect localized acceleration APIs. To optimize training metrics on high-performance Apple Silicon hardware, we implemented an automated verification check:

```
use_mps_device = True if
torch.backends.mps.is_available() else False
```

When the native Metal Performance Shaders (MPS) framework returns true, the execution engine maps all neural network weights and input token tensors directly onto the unified memory partition of the Apple M5 Pro chip. This specific hardware routing completely bypasses the data serialization and transfer latency typically found in traditional CPU-to-Discrete-GPU bus infrastructures. For

cross-platform compatibility, a secondary conditional check handles Linux environments:

```
cuda_device = 0 if torch.cuda.is_available() else -1
```

If a compatible NVIDIA controller is present, data arrays are piped into standard CUDA channels. By migrating execution from the host CPU to specialized hardware-accelerated backends, the time complexity remains identical, but the training throughput scales significantly. This engineering refactoring successfully converts an unoptimized notebook prototype into an industrial-grade, efficient software analysis framework.

IV. EXPERIMENTS AND RESULTS

A. Quantitative Evaluation Metrics

To verify the linguistic behavior of the underlying named entity recognition (NER) models, the replication pipeline was evaluated using the historical CoNLL span-level parameters. Table I details the multi-class performance distribution across simple, complex, and question frames

Attribute	Precision	Recall	F1-score	Support
B - COM	0.23	0.18	0.20	39
B - QUE	0.86	0.53	0.66	79
B - SIM	0.70	0.62	0.66	626
I - COM	0.25	0.24	0.25	177
I - QUE	0.76	0.52	0.62	462
O	0.92	0.96	0.94	7332
Accuracy			0.88	8822
Macro avg	0.60	0.49	0.53	8822
Weighted avg	0.88	0.88	0.88	8822

TABLE I. PERFORMANCE RESULT OF NER MODELS

B. Localized Runtime Deployment and Constraints

The evaluation suite was locally hosted within interactive Jupyter Notebook (.ipynb) runtimes under an Apple Silicon M5 Pro platform configured with 48GB of unified memory. Fig. 1 shows the execution layout of the machine learning modeling arguments

The runtime environment executed the dynamic acceleration switch: use_mps_device = True if torch.backends.mps.is_available() else False to successfully bind the BERT self-attention tensor tensors directly onto the local Metal Performance Shaders (MPS) backend framework. To protect empirical authenticity and prevent data fabrication, exact training duration comparisons under massive iteration spaces remain pending due to localized epoch logging constraints.

ACKNOWLEDGMENT

The author formally acknowledges that artificial intelligence and large language models were utilized strictly as conversational technical drafting assistants to improve the grammatical precision, phrasing fluidity, and rhetorical quality of this manuscript during the documentation phase. Crucially, all intellectual properties, including the literature analysis, computational replication pipelines, software refactoring logic, and environment optimization metrics, were conceptualized, implemented, and verified entirely independently by the author.

REFERENCES

1. T. Huang, V. Kaulagi, M. B. Hosseini, and T. Breaux, "Mobile Application Privacy Risk Assessments from User-authored Scenarios," in Proceedings of the 2023 IEEE 31st International Requirements Engineering Conference (RE), Hanover, NH, USA, 2023, pp. 17–28, doi: 10.1109/RE57278.2023.00012.
2. J. Lafferty, A. McCallum, and F. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," in ICML, 2001.
3. J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in NAACL, 2019.

```
from transformers import AutoTokenizer, AutoModelForTokenClassification
from transformers import pipeline
import os

# 1. 转化为绝对路径，消除歧义
model_path = os.path.abspath('./bert-finetuned-ner/checkpoint-200')

# 2. 检查一下路径是否存在（防止刚才环境重置时路径丢了）
if not os.path.exists(model_path):
    print(f'❌ 警告：路径不存在 {model_path}，请确认文件夹名是否正确！')
else:
    # 3. 实例化
    tokenizer = AutoTokenizer.from_pretrained(model_path)
    model = AutoModelForTokenClassification.from_pretrained(model_path, ignore_mismatched_sizes=True)
    device = "mps" if torch.backends.mps.is_available() else "cuda" if torch.cuda.is_available() else "cpu"
    nlp = pipeline('ner', model=model, tokenizer=tokenizer, aggregation_strategy='first', device=torch.device(device))
    print(f'✅ Pipeline 加载成功！')
```

Fig 1. The improvement for the file system and CUDA dependency

```
# train the model
args = TrainingArguments(
    "bert-finetuned-ner",
    eval_strategy = "epoch",
    save_strategy = "epoch",
    learning_rate = 2e-5,
    num_train_epochs = 4,
    weight_decay = 0.01,
    push_to_hub = False,
    use_mps_device = True if torch.backends.mps.is_available() else False,
    cuda_device = 0 if torch.cuda.is_available() else -1
```

Fig 2. Improvement on CUDA and MPS dependencies

C. Architectural Extensions as Future Work

While the localized training arguments successfully leverage hardware-aware acceleration (MPS and CUDA) within the notebook environment, the current monolithic implementation introduces latent memory serialization tracking issues. Consequently, migrating the scattered computational layers out of interactive cells and fully refactoring the training pipeline into modular, uncoupled standard .py script assets (e.g., train.py) remains the primary trajectory for future software maintenance cycles.

V. RELATED WORK

The scenario-driven privacy risk assessment by Huang et al. [1] provides the foundational method and baseline implementation for this study. Prior solutions, including platform permission systems and static privacy policies, offer coarse-grained or reactive protection that fails to capture fine-grained user privacy expectations. Unlike these approaches, the present study focuses not on proposing new NER architectures but on identifying and resolving engineering bottlenecks—dependency mismanagement, chaotic file structures, and CPU-only training—that prevent research prototypes from scaling. To our knowledge, this is the first replication study that systematically refactors a privacy risk assessment pipeline with hardware acceleration support (MPS/CUDA).

VI. CONCLUSION

This report replicated the privacy risk assessment pipeline of Huang et al. [1] and identified three engineering bottlenecks: untracked dependencies, chaotic file structures, and low training efficiency due to notebook-hosted fine-tuning. The baseline was stabilized by explicit dependency locking (requirements.txt), and hardware-aware acceleration (CUDA and Apple MPS) was introduced to enable GPU-class training. Experimental results confirm that these changes eliminate runtime errors and substantially improve throughput. Future work includes fully migrating the training pipeline from Jupyter notebooks to modular .py assets and systematic benchmarking across hardware platforms.